

Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	M. Tech CSE [5 Years Integrated]	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	Due date: 8-08-2025

Experiment 3: Email Spam or Ham Classification using Naïve Bayes, KNN, and SVM

Aim:

To classify emails as spam or ham using three classification algorithms—Naïve Bayes, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM)—and evaluate their performance using accuracy metrics and K-Fold cross-validation.

Libraries Used:

1. pandas
2. numpy
3. matplotlib
4. seaborn
5. `from sklearn.model_selection import train_test_split, cross_val_score, KFold`
6. `from sklearn.preprocessing import StandardScaler`
7. `from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB`
8. `from sklearn.neighbors import KNeighborsClassifier`
9. `from sklearn.metrics import classification_report, confusion_matrix`
10. `from sklearn.metrics import roc_auc_score, ConfusionMatrixDisplay`
11. google.colab.drive

Python Code with Output

1. Load and Preprocess the Dataset:

CODE:

```
# Load dataset
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix,
roc_auc_score, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import drive
drive.mount('/content/drive')
# Load your dataset
df = pd.read_csv("/content/drive/MyDrive/spam_a.csv")
print("Initial Data Shape:", df.shape)
print(df.head())
# Load dataset
df = pd.read_csv("/content/drive/MyDrive//spam_a.csv")
```

2. EDA:

• Handling Missing Values:

CODE:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Features and labels
X = df.drop("class", axis=1)
y = df["class"]

# Check for missing values
missing_values = df.isnull().sum().sum()

# Separate features and labels
X = df.drop(columns=['class'])
y = df['class']

# Normalize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
missing_values
```

OUTPUT:

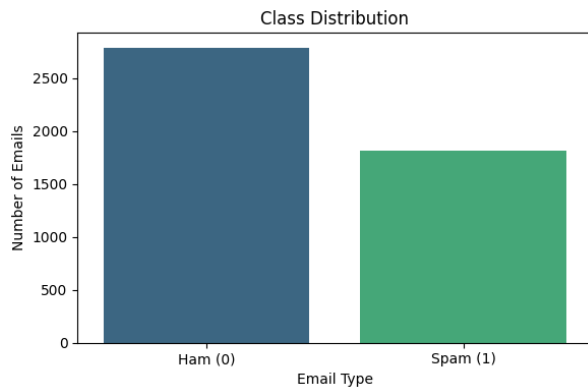
```
np.int64(0)
```

• Class Distribution:

CODE:

```
class_counts = y.value_counts()
# Plot class balance
plt.figure(figsize=(6, 4))
sns.barplot(x=class_counts.index, y=class_counts.values, palette='viridis')
plt.xticks([0, 1], ['Ham (0)', 'Spam (1)'])
plt.title('Class Distribution')
plt.ylabel('Number of Emails')
plt.xlabel('Email Type')
plt.tight_layout()
plt.show()
```

OUTPUT:

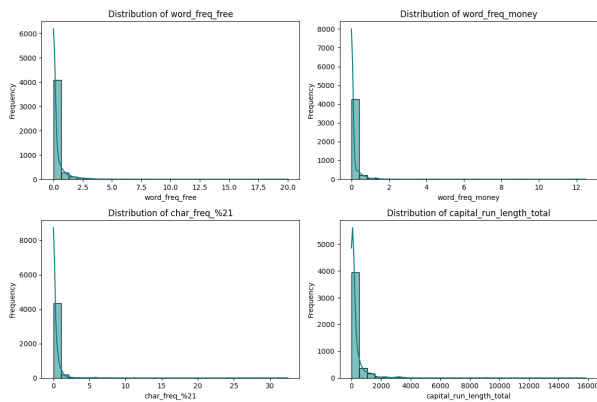


• Feature Distribution:

CODE:

```
selected_features = ['word_freq_free', 'word_freq_money', 'char_freq_%21',
                    'capital_run_length_total']
plt.figure(figsize=(12, 8))
for i, feature in enumerate(selected_features):
    plt.subplot(2, 2, i + 1)
    sns.histplot(df[feature], bins=30, kde=True, color='teal')
    plt.title(f'Distribution of {feature}')
    plt.xlabel(feature)
    plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

OUTPUT:



3. Train-Test Split Dataset:

CODE:

```
from sklearn.model_selection import train_test_split

# Use the normalized dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

print("Train set size:", X_train.shape)
print("Test set size:", X_test.shape)
```

OUTPUT:

```
Train set size: (3680, 57)
Test set size: (921, 57)
```

4. Train Models:

A) Naïve Bayes:

CODE:

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import time
```

```

# ----- 1. Prepare Scaled and Raw Versions -----
X_raw = df.drop('class', axis=1)
y = df['class']

# Scaled version for GaussianNB & KNN
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)

# Train-test split (same random state for consistency)
X_train_scaled, X_test_scaled, y_train, y_test = train_test_split(
X_scaled, y, test_size=0.2, stratify=y, random_state=42)
X_train_raw, X_test_raw, _, _ = train_test_split( X_raw, y, test_size=0.2,
stratify=y, random_state=42)

# ----- 2. Naïve Bayes Models -----
nb_results = []

# GaussianNB (use scaled data)
gnb = GaussianNB()
gnb.fit(X_train_scaled, y_train)
y_pred = gnb.predict(X_test_scaled)

print("\nGaussianNB Classification Report:\n")
print(classification_report(y_test, y_pred, digits=4))
nb_results.append({
    "Model": "GaussianNB",
    "Accuracy": gnb.score(X_test_scaled, y_test),
    "Precision": classification_report(y_test, y_pred, output_dict=True)['1']['precision'],
    "Recall": classification_report(y_test, y_pred, output_dict=True)['1']['recall'],
    "F1 Score": classification_report(y_test, y_pred, output_dict=True)['1']['f1-score']
})

# MultinomialNB (use raw data)
mnb = MultinomialNB()
mnb.fit(X_train_raw, y_train)
y_pred = mnb.predict(X_test_raw)
print("\nMultinomialNB Classification Report:\n")
print(classification_report(y_test, y_pred, digits=4))
nb_results.append({
    "Model": "MultinomialNB",
    "Accuracy": mnb.score(X_test_raw, y_test),
    "Precision": classification_report(y_test, y_pred, output_dict=True)['1']['precision'],
    "Recall": classification_report(y_test, y_pred, output_dict=True)['1']['recall'],
    "F1 Score": classification_report(y_test, y_pred, output_dict=True)['1']['f1-score']
})

```

```

# BernoulliNB (use raw data)
bnb = BernoulliNB()
bnb.fit(X_train_raw, y_train)
y_pred = bnb.predict(X_test_raw)
print("\nBernoulliNB Classification Report:\n")
print(classification_report(y_test, y_pred, digits=4))
nb_results.append({
    "Model": "BernoulliNB",
    "Accuracy": bnb.score(X_test_raw, y_test),
    "Precision": classification_report(y_test, y_pred, output_dict=True)['1']['precision'],
    "Recall": classification_report(y_test, y_pred, output_dict=True)['1']['recall'],
    "F1 Score": classification_report(y_test, y_pred, output_dict=True)['1']['f1-score']
})

```

OUTPUT:

GaussianNB Classification Report:				
	precision	recall	f1-score	support
0	0.9654	0.7509	0.8448	558
1	0.7146	0.9587	0.8188	363
accuracy			0.8328	921
macro avg	0.8400	0.8548	0.8318	921
weighted avg	0.8666	0.8328	0.8345	921
MultinomialNB Classification Report:				
	precision	recall	f1-score	support
0	0.8121	0.8208	0.8164	558
1	0.7199	0.7080	0.7139	363
accuracy			0.7763	921
macro avg	0.7660	0.7644	0.7651	921
weighted avg	0.7757	0.7763	0.7760	921
BernoulliNB Classification Report:				
	precision	recall	f1-score	support
0	0.8788	0.9229	0.9003	558
1	0.8716	0.8044	0.8367	363
accuracy			0.8762	921
macro avg	0.8752	0.8637	0.8685	921
weighted avg	0.8760	0.8762	0.8753	921

B) K-Nearest Neighbors (k=5 default, KDTree/BallTree is used automatically):

CODE:

```

#3. KNN { Varying k
k_values = [1, 3, 5, 7]
knn_results = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)

    print(f"\nKNN (k={k}) Classification Report:\n")
    print(classification_report(y_test, y_pred, digits=4))
    knn_results.append({
        "k": k,
        "Accuracy": knn.score(X_test_scaled, y_test),

```

```

    "Precision":
    classification_report(y_test, y_pred, output_dict=True)['1']['precision'],
    "Recall":
    classification_report(y_test, y_pred, output_dict=True)['1']['recall'],
    "F1 Score":
    classification_report(y_test, y_pred, output_dict=True)['1']['f1-score']
    })
# 4. KNN with KDTree vs BallTree
for algorithm in ['kd_tree', 'ball_tree']:
    knn = KNeighborsClassifier(n_neighbors=5, algorithm=algorithm)
    start = time.time()
    knn.fit(X_train_scaled, y_train)
    elapsed = time.time() - start
    y_pred = knn.predict(X_test_scaled)

    print(f"\nKNN with {algorithm.upper()} Report:\n")
    print(classification_report(y_test, y_pred, digits=4))
    print(f"Training time: {elapsed:.4f} seconds\n")

```

OUTPUT:

KNN (k=1) Classification Report:					
	precision	recall	f1-score	support	
0	0.9113	0.9211	0.9162	558	
1	0.8768	0.8623	0.8694	363	
accuracy			0.8979	921	
macro avg	0.8940	0.8917	0.8928	921	
weighted avg	0.8977	0.8979	0.8978	921	

KNN (k=3) Classification Report:					
	precision	recall	f1-score	support	
0	0.9133	0.9247	0.9190	558	
1	0.8820	0.8650	0.8734	363	
accuracy			0.9012	921	
macro avg	0.8976	0.8949	0.8962	921	
weighted avg	0.9010	0.9012	0.9010	921	

KNN (k=5) Classification Report:					
	precision	recall	f1-score	support	
0	0.9168	0.9283	0.9225	558	
1	0.8876	0.8705	0.8790	363	
accuracy			0.9055	921	
macro avg	0.9022	0.8994	0.9008	921	
weighted avg	0.9053	0.9055	0.9054	921	

KNN (k=7) Classification Report:					
	precision	recall	f1-score	support	
0	0.9156	0.9337	0.9246	558	
1	0.8949	0.8678	0.8811	363	
accuracy			0.9077	921	
macro avg	0.9053	0.9007	0.9028	921	
weighted avg	0.9075	0.9077	0.9074	921	

KNN with KD_TREE Report:					
	precision	recall	f1-score	support	
0	0.9168	0.9283	0.9225	558	
1	0.8876	0.8705	0.8790	363	
accuracy			0.9055	921	
macro avg	0.9022	0.8994	0.9008	921	
weighted avg	0.9053	0.9055	0.9054	921	

Training time: 0.0247 seconds

KNN with BALL_TREE Report:					
	precision	recall	f1-score	support	
0	0.9168	0.9283	0.9225	558	
1	0.8876	0.8705	0.8790	363	
accuracy			0.9055	921	
macro avg	0.9022	0.8994	0.9008	921	
weighted avg	0.9053	0.9055	0.9054	921	

Training time: 0.0195 seconds

C) Support Vector Machine (SVM):

CODE:

```
from sklearn.svm import SVC
svm_results = []
kernels = ['linear', 'poly', 'rbf', 'sigmoid']

# Store fitted models in variables for later use
svm_model_linear = None
svm_model_poly = None
svm_model_rbf = None
svm_model_sigmoid = None
for kernel in kernels:
    print(f"\nTraining SVM with {kernel.upper()} kernel ")
    svm_model = SVC(kernel=kernel, random_state=42, probability=True)
    start_time = time.time()    svm_model.fit(X_train_scaled, y_train)
    end_time = time.time()
    training_time = end_time - start_time

    y_pred = svm_model.predict(X_test_scaled)
    report = classification_report(y_test, y_pred, digits=4, output_dict=True)

    svm_results.append({
        "Kernel": kernel.upper(),
        "Accuracy": report['accuracy'],
        "Precision": report['1']['precision'],
        "Recall": report['1']['recall'],
        "F1 Score": report['1']['f1-score'],
        "Training Time (s)": training_time
    })

print(f"Classification Report for {kernel.upper()} kernel:")
print(classification_report(y_test, y_pred, digits=4))
print(f"Training Time: {training_time:.4f} seconds")

# Assign the fitted model to the corresponding variable
if kernel == 'linear':
    svm_model_linear = svm_model
elif kernel == 'poly':
    svm_model_poly = svm_model
elif kernel == 'rbf':
    svm_model_rbf = svm_model
elif kernel == 'sigmoid':
    svm_model_sigmoid = svm_model
```


OUTPUT:

```
Training SVM with LINEAR kernel
Classification Report for LINEAR kernel:
              precision    recall  f1-score   support

    0       0.9349       0.9516       0.9432         558
    1       0.9235       0.8981       0.9106         363

 accuracy       0.9305         921
 macro avg       0.9292       0.9248       0.9269         921
weighted avg       0.9304       0.9305       0.9303         921
```

Training Time: 3.9782 seconds

```
Training SVM with POLY kernel
Classification Report for POLY kernel:
              precision    recall  f1-score   support

    0       0.7376       0.9875       0.8444         558
    1       0.9598       0.4601       0.6220         363

 accuracy       0.7796         921
 macro avg       0.8487       0.7238       0.7332         921
weighted avg       0.8252       0.7796       0.7568         921
```

Training Time: 3.7709 seconds

```
Training SVM with RBF kernel
Classification Report for RBF kernel:
              precision    recall  f1-score   support

    0       0.9270       0.9552       0.9409         558
    1       0.9277       0.8843       0.9055         363

 accuracy       0.9273         921
 macro avg       0.9274       0.9197       0.9232         921
weighted avg       0.9273       0.9273       0.9269         921
```

Training Time: 2.1381 seconds

```
Training SVM with SIGMOID kernel
Classification Report for SIGMOID kernel:
              precision    recall  f1-score   support

    0       0.9034       0.9050       0.9042         558
    1       0.8536       0.8512       0.8524         363

 accuracy       0.8838         921
 macro avg       0.8785       0.8781       0.8783         921
weighted avg       0.8838       0.8838       0.8838         921
```

Training Time: 1.8816 seconds

5. Performance Analysis [Confusion Matrix and ROC for Each]:

CODE:

```
def plot_conf_matrix_and_roc(model, X, y_true, title="Model"):
    y_pred = model.predict(X)
    y_proba = model.predict_proba(X)[:, 1]

    # Confusion Matrix
    cm = confusion_matrix(y_true, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    disp.plot()
    plt.title(f"Confusion Matrix: {title}")
    plt.show()

    # ROC Curve
    fpr, tpr, _ = roc_curve(y_true, y_proba)
    roc_auc = auc(fpr, tpr)
    plt.figure()
    plt.plot(fpr, tpr, label=f"{title} (AUC = {roc_auc:.4f})")
    plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curve")
    plt.legend(loc="lower right")
    plt.grid()
    plt.show()

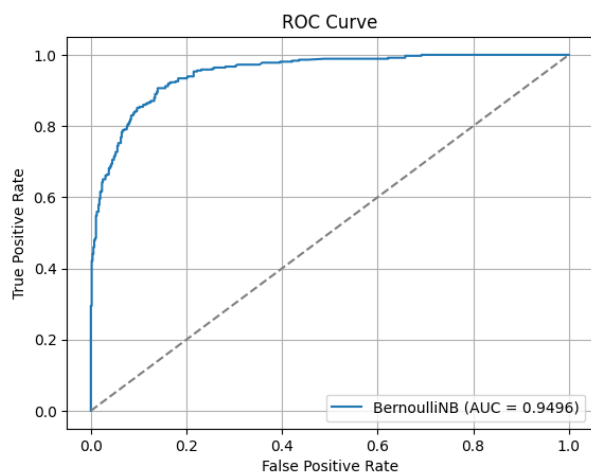
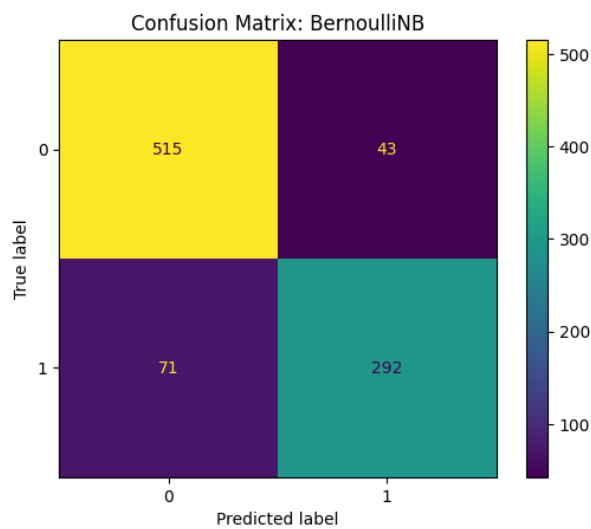
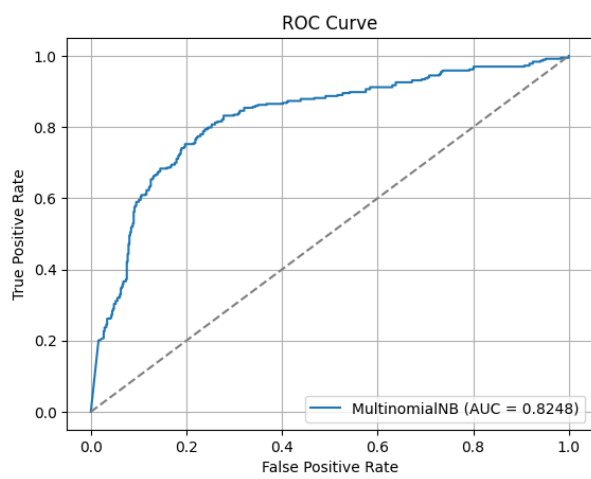
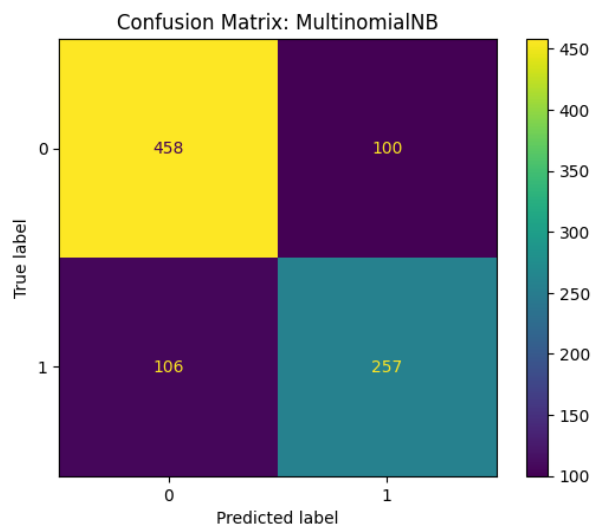
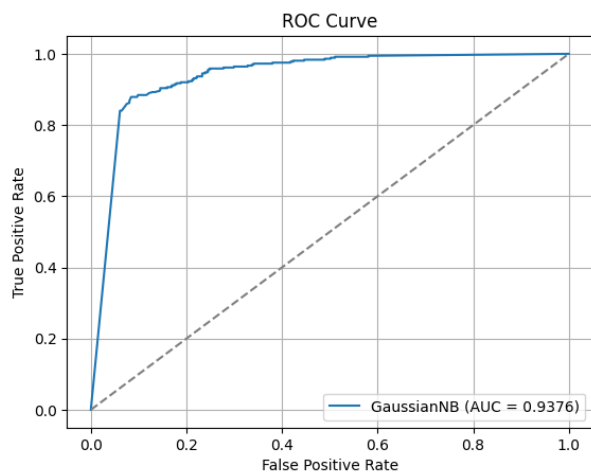
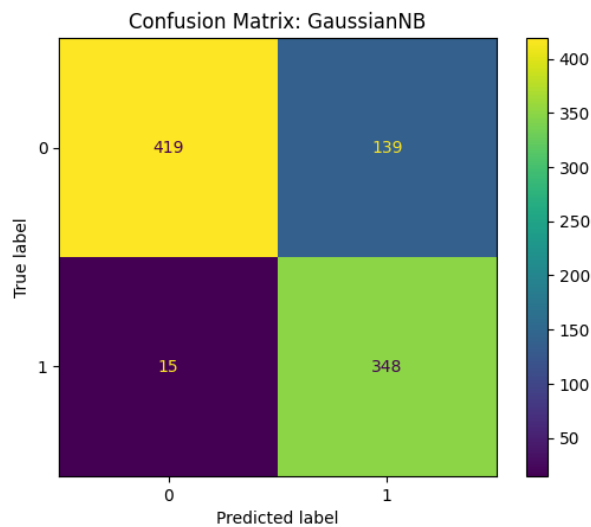
# Naïve Bayes
plot_conf_matrix_and_roc(gnb, X_test_scaled, y_test, "GaussianNB")
plot_conf_matrix_and_roc(mnb, X_test_raw, y_test, "MultinomialNB")
plot_conf_matrix_and_roc(bnb, X_test_raw, y_test, "BernoulliNB")

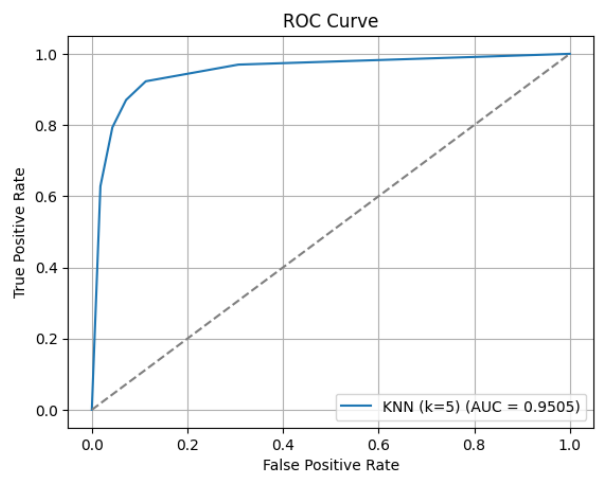
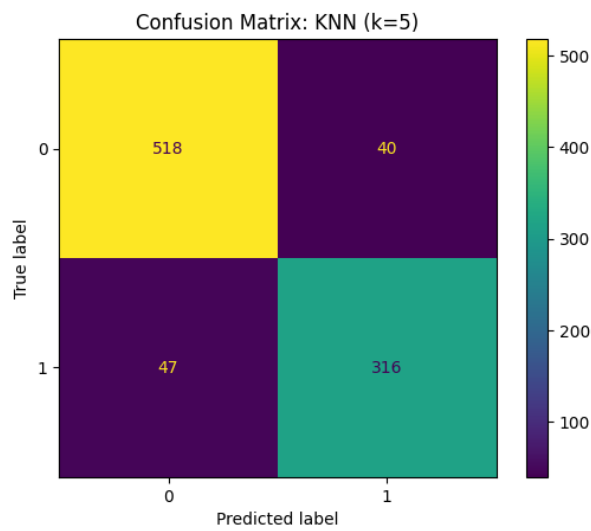
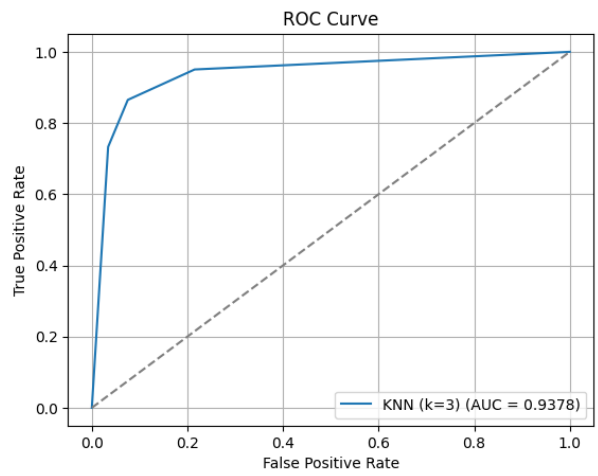
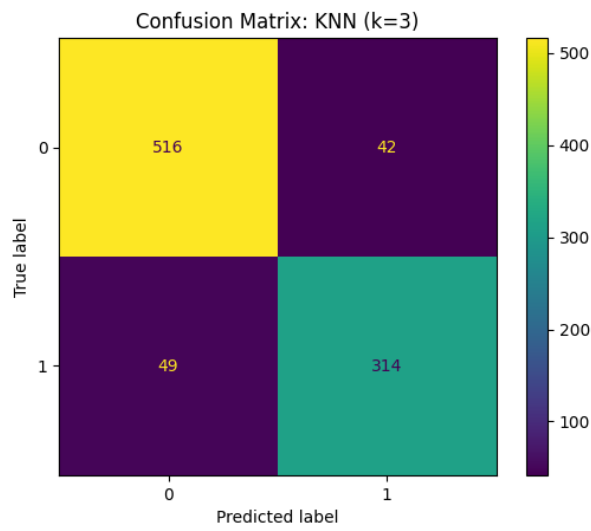
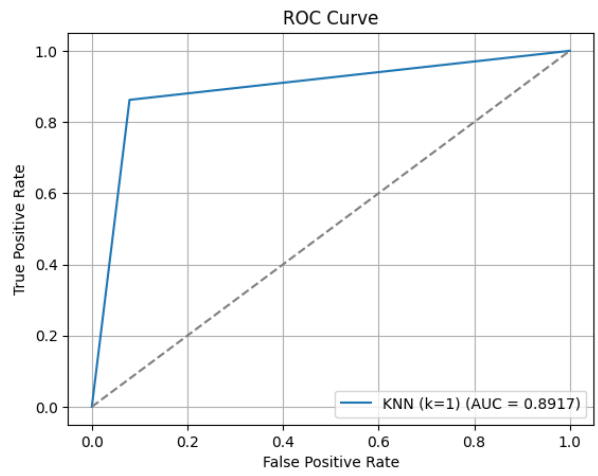
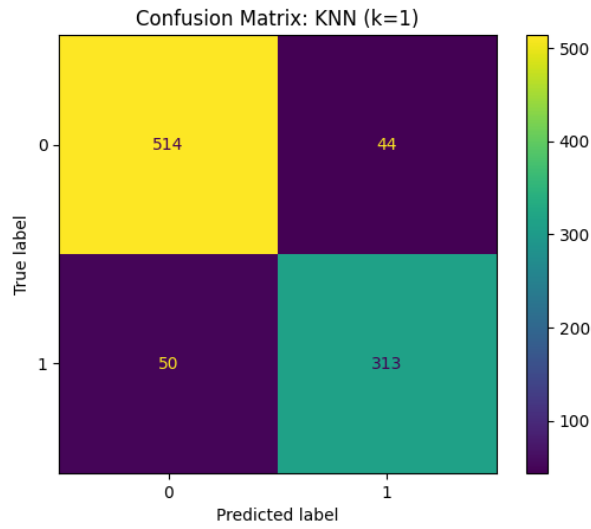
# KNN: Different k values
for k in [1, 3, 5, 7]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    plot_conf_matrix_and_roc(knn, X_test_scaled, y_test, f"KNN (k={k})")

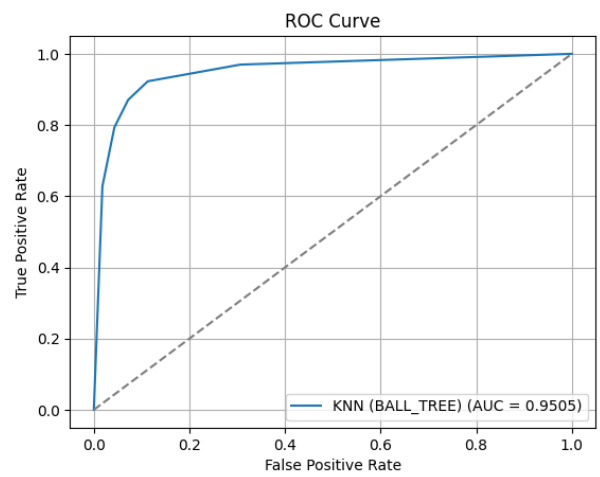
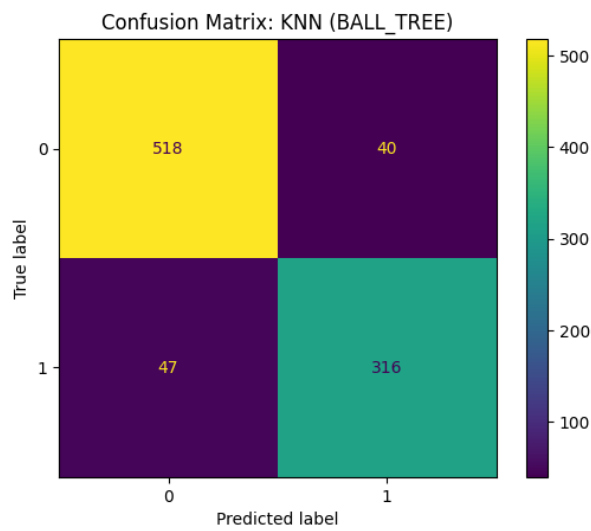
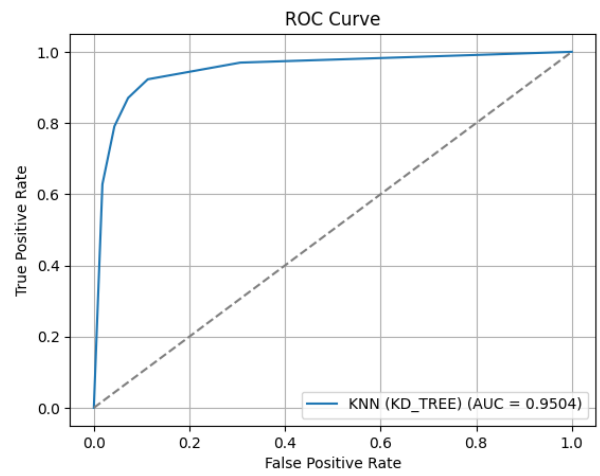
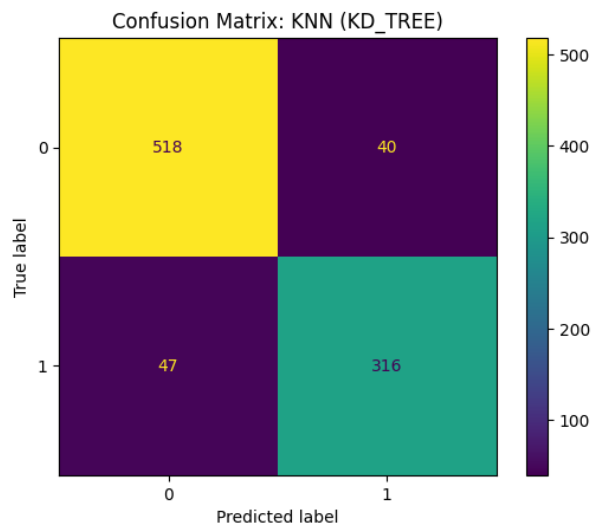
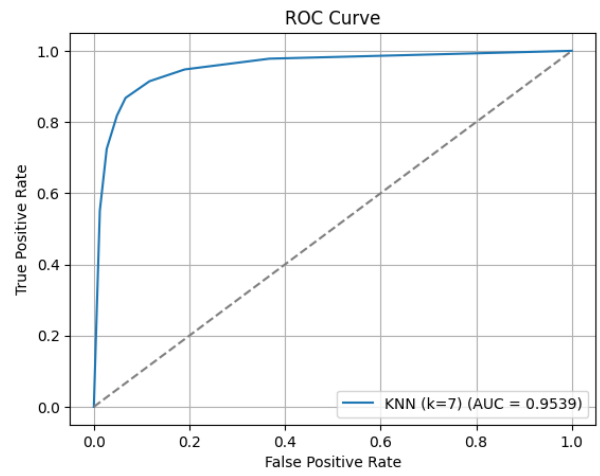
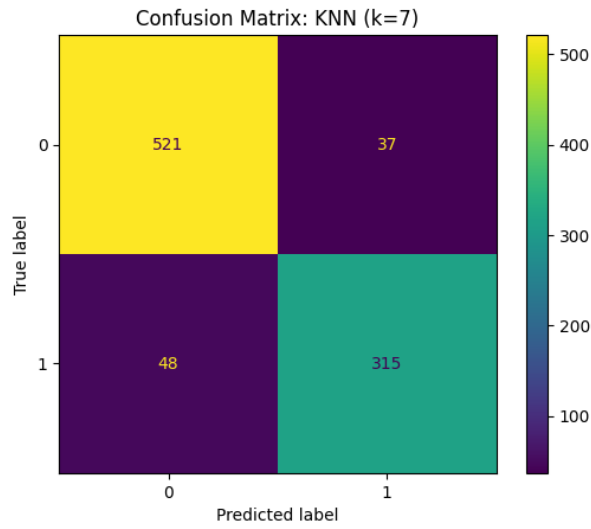
# KNN: KDTree and BallTree
for algo in ['kd_tree', 'ball_tree']:
    knn_tree = KNeighborsClassifier(n_neighbors=5, algorithm=algo)
    knn_tree.fit(X_train_scaled, y_train)
    plot_conf_matrix_and_roc(knn_tree, X_test_scaled, y_test, f"KNN ({algo.upper()}")

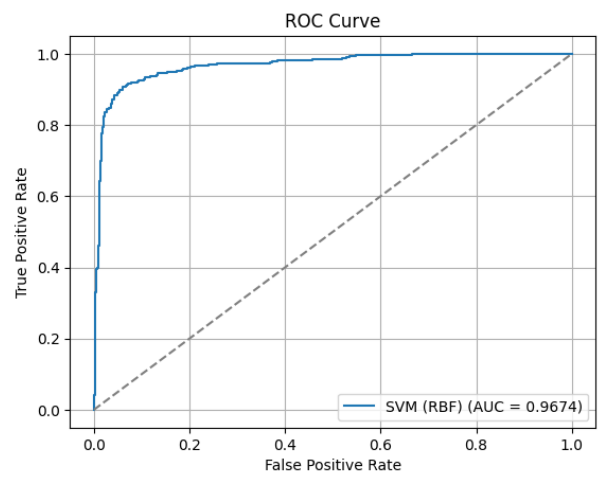
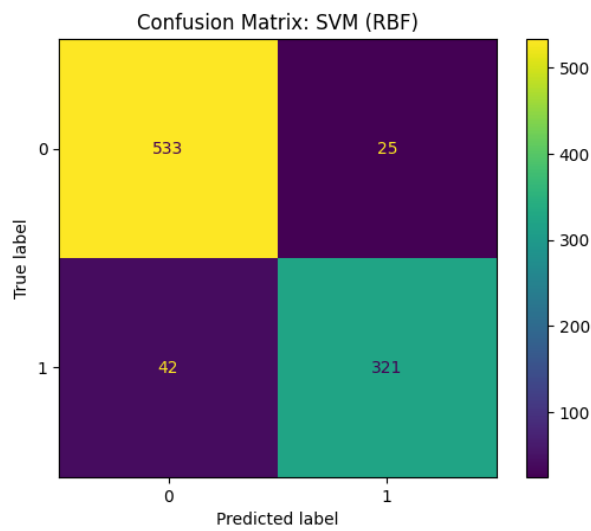
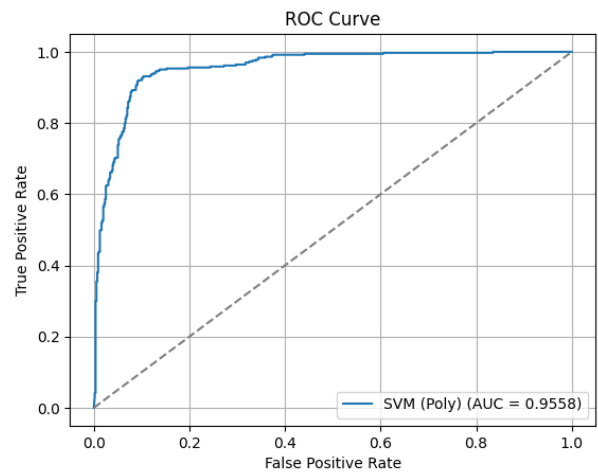
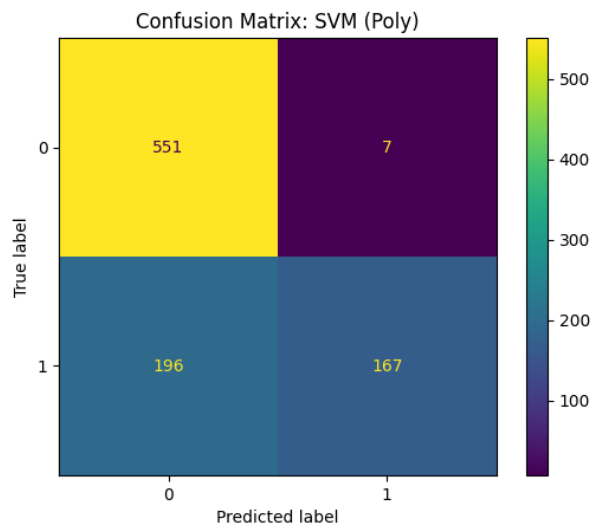
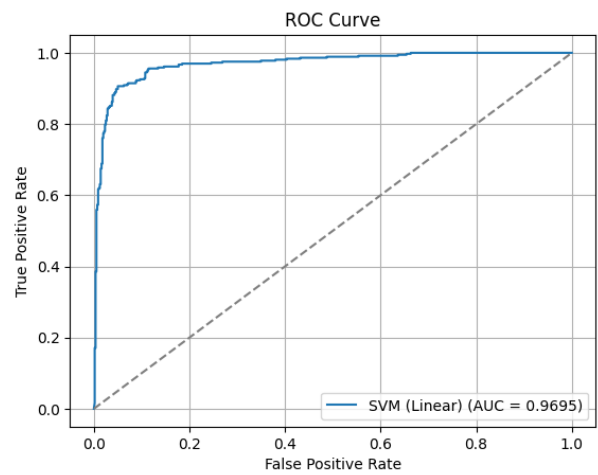
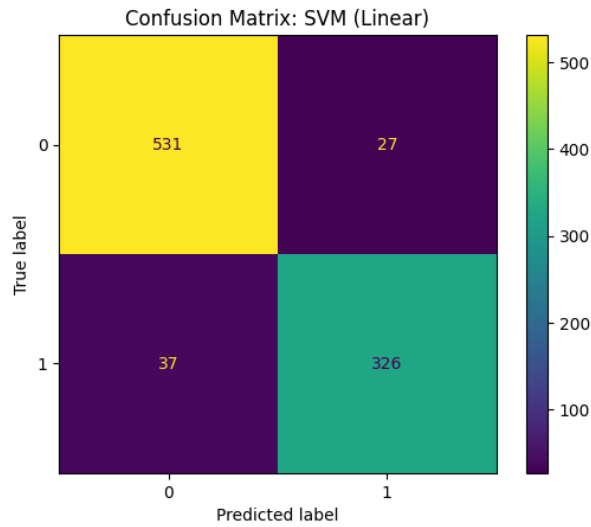
# Plot ROC and AUC for each trained SVM model
plot_conf_matrix_and_roc(svm_model_linear, X_test_scaled, y_test, "SVM (Linear)")
plot_conf_matrix_and_roc(svm_model_poly, X_test_scaled, y_test, "SVM (Poly)")
plot_conf_matrix_and_roc(svm_model_rbf, X_test_scaled, y_test, "SVM (RBF)")
plot_conf_matrix_and_roc(svm_model_sigmoid, X_test_scaled, y_test, "SVM (Sigmoid)")
```

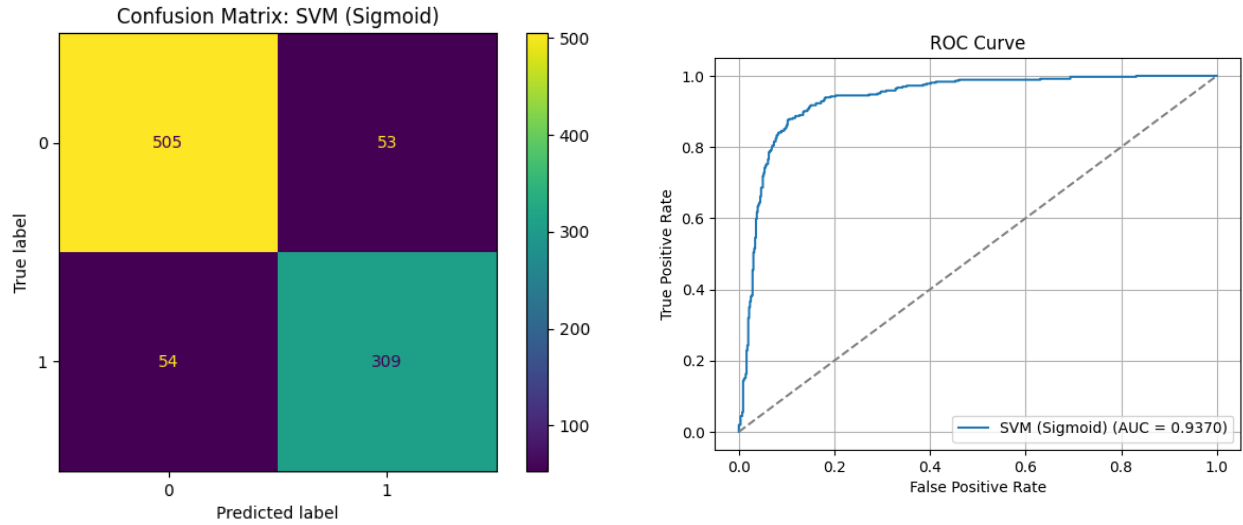
OUTPUT:











6. K-Fold Cross-Validation (k=5)

- Table 1: Naive Bayes Variant Comparison

Table 1: Naive Bayes Variant Comparison

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8328	0.7763	0.8762
Precision	0.8666	0.7757	0.8760
Recall	0.8328	0.7763	0.8762
F1 Score	0.8345	0.7760	0.8753

- Table 2: KNN with Varying k

Table 2: KNN with Varying k

k	Accuracy	Precision	Recall	F1 Score
1	0.8940	0.8977	0.8979	0.8978
3	0.8976	0.9010	0.9012	0.9010
5	0.9055	0.9053	0.9055	0.9054
7	0.9077	0.9075	0.9077	0.9074

- Table 3: KNN: KDTree vs BallTree

Table 3: KDTree vs BallTree

Metric	KDTree	BallTree
Accuracy	0.9055	0.9055
Precision	0.9053	0.9053
Recall	0.9055	0.9055
F1 Score	0.9054	0.9054
Training Time (s)	0.0207	0.0126

- **Table 4: SVM Kernel-wise Results**

Table 4: SVM Kernel Comparison

Kernel	Hyperparameters	Accuracy	Precision	Recall	F1 Score	Time(s)
Linear	C=1	0.9305	0.9235	0.8981	0.9303	3.6872s
Polynomial	C=1, degree=3, gamma=0.01	0.7796	0.9598	0.4600	0.7568	3.0892s
RBF	C=1, gamma=0.01	0.9273	0.9277	0.8843	0.9269	1.7163s
Sigmoid	C=1, gamma=0.01	0.8838	0.8535	0.8512	0.8838	1.8344s

- **Table 5: K-Fold Cross-Validation Results (K = 5)**

Table 5: K-Fold Cross Validation

Fold	NB (Gaussian)	NB (Multinomial)	NB (Bernoulli)	KNN (k=7)	SVM (Linear)
Fold 1	0.8219	0.7861	0.8806	0.8958	0.9251
Fold 2	0.8033	0.7761	0.8902	0.9130	0.9283
Fold 3	0.7946	0.7804	0.8837	0.9130	0.9163
Fold 4	0.8228	0.8141	0.8869	0.9000	0.9370
Fold 5	0.8337	0.8011	0.8902	0.9120	0.9304
Average	0.8153	0.7916	0.8863	0.9068	0.9274

Best Practices:

- Always handle missing values and normalize features before training models to ensure better accuracy and stability.
- Experiment with different variants of algorithms (e.g., Gaussian, Multinomial Naïve Bayes; Linear, RBF kernels for SVM) to identify the most suitable one for the dataset.
- Use techniques such as Grid Search or Random Search to optimize hyperparameters like k in KNN or C and gamma in SVM.
- Apply K-Fold Cross-Validation to evaluate model performance reliably and reduce overfitting.
- Use confusion matrices and ROC curves to interpret classification performance visually and make informed decisions.

Learning Outcomes:

1. Learned how to preprocess email datasets, including handling missing values and normalization for better model performance.
2. Able to implement and compare multiple classification algorithms (Naïve Bayes, KNN, SVM) for a real-world spam detection problem.
3. Understood the significance of evaluation metrics such as Accuracy, Precision, Recall, and F1-score in assessing model performance.
4. Applied K-Fold Cross-Validation for robust model evaluation and avoiding bias in performance measurement.
5. Learned to visualize and interpret model results using Confusion Matrix and ROC Curve for better understanding of classification outcomes.